

Name:
Z number:

Total Points:15

COP 3410-001: Data Strc Algrthm Anlys

Python Florida Atlantic University

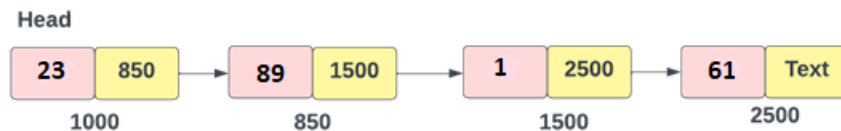
Linked List Worksheet

NOTE:

- Each question carries 1 point.
- All solutions must be answered in the space labeled “put your solutions here.”
- submit the completed worksheet in the form of a PDF.
- The file name must be *YourName_Linked List Worksheet.pdf*

LINKED LIST

- i. In the given image, what will be the output when accessing the data attribute of the statement “head.next.data”
ii. After executing the code "temp = temp.next" two times, with the initial value of temp set to head, which node's data will be printed from the given image?



Put Your Solution Here:

-
-

2. Which of the following four options best suits the operations of the two statements below?

```
new_node.next = head  
head = new_node
```

- a. Create a new node for a linked list.
- b. Insert a new node to the front of a linked list.
- c. Delete a new node from a linked list.
- d. Insert a new node to the end of the linked list.

Put your option here:

3. What is the time complexity?
- a. to count the number of elements in the linked list?
 - b. to perform reversal operation in the linked list.

Put Your Solution Here:

- a.
 - b.
-

4. Treat this as an algorithm pseudocode, Now complete the function `delete_a_node(self,data)` which will delete the node inside the linked list.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def delete_a_node(self,data):
        """ write the code here to delete a node inside the linkedlist """
```

Assume your linked list is 1-->23 --> 3 -->56 -->41 -->9. Now delete the node 56.

Put Your Solution Here:

5. Choose the best option suitable and justify the selected option.
Doubly Linked List Consists of:

- a. 1-Data, 1- Address
- b. 2-Data, 1- Address
- c. 1-Data, 0- Address
- d. 1-Data, 2- Address

Put Your Solution Here:

Justification:

-
6. What could be the best and worst-case time complexity for searching for a value in a given single Linked List? choose the correct option and justify your answer.
- a. $O(1)$ and $O(\log n)$
 - b. $O(1)$ and $O(1)$
 - c. $O(1)$ and $O(n^2)$
 - d. $O(1)$ and $O(n)$

Put Your Solution Here:

Justification:

7. Select the code snippet that best suits and correctly represents the following statement:
"Assign the newly created node's 'next' pointer to the current head reference, so that it points to the address of the previous starting node of the linked list."

- a. `head.next = new_node`
- b. `new_node.next = head`
- c. `head = new_node`
- d. `new_node = head`

Put Your Solution Here:

-
8. How do you remove a node with a specific value 'val' from a singly linked list in Python?
- a) Using a loop to find the node and then remove it directly.
 - b) Changing the value of the node to None.
 - c) using a loop and pointing the previous node's next to the next node.
 - d) Assigning 'val' to the current node's next

Put Your Solution Here:

9. Write True or False for the following statements:
- a. For a single Linked list, allow constant time insertion and deletion at both the beginning and the end of the list.
 - b. Linked lists have constant time complexity for accessing elements at any position.
 - c. In a doubly linked list, each node contains references to both the next and previous nodes.
 - d. To delete a node in a singly linked list, you can delete the node by updating the next pointer of the previous node and the node to be deleted.
 - e. In a singly linked list, each node contains a reference to the next node but not to the previous node.
 - f. Linked lists are an example of a linear data structure.

Put Your Solution Here:

- a.
 - b.
 - c.
 - d.
 - e.
 - f.
-

10. What will be the output of the following code snippet for the linked list:
[10, 20, 30, 40, 50,60,70]?

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def solve(start):
    if start is None:
        return
    print(start.data, end=" ")
    if start.next is not None:
        solve(start.next.next)
    print(start.data, end=" ")

start_node = Node(10)
start_node.next = Node(20)
start_node.next.next = Node(30)
start_node.next.next.next = Node(40)
start_node.next.next.next.next = Node(50)
start_node.next.next.next.next.next = Node(60)
start_node.next.next.next.next.next.next = Node(70)
# Call the function to print the linked list data in the specified pattern
solve(start_node)
```

Put Your Solution Here:

11. Tell in words what the following code snippet is planning to return.

```
def fun(head):  
    # Initialize two pointers, runner and walker, both pointing to the head of the linked list.  
    runner = head  
    walker = head  
  
    # Continue the loop until runner reaches the end or the node before the end.  
    while runner.next is not None and runner.next.next is not None:  
        # Move runner two steps ahead.  
        runner = runner.next.next  
        # Move walker one step ahead.  
        walker = walker.next  
  
    return walker.data
```

Put Your Solution Here:

12. fill the below Python function `insert_at_tail(self, x)` for the `List` class that inserts a `Node` object `x` at the *tail* of the doubly circular list.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyCircularList:
    def __init__(self):
        self.head = None

    def insert_at_tail(self, x):
        """
        Fill the incomplete code
        """

# Example usage:
# Create a doubly circular list
dcl = DoublyCircularList()

# Insert Node objects at the tail
node1 = Node(10)
node2 = Node(20)
node3 = Node(30)

dcl.insert_at_tail(node1)
dcl.insert_at_tail(node2)
dcl.insert_at_tail(node3)
```

Put Your Solution Here:

Florida Atlantic University

13. Complete the following functions for the LinkedList class:

"add_first()": This function should add a new element at the beginning of the linked list.

"find_size()": This function should calculate the size of the singly linked list.

"search()": This function should search for a specific item in the singly linked list and return true if the item is found, otherwise, return false.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def add_first(self, data):
        """
        Complete the function.
        """

    def get_size(self):
        """
        Complete the function.
        """

    def search(self, item):
        """
        Complete the function.
        """

# Example usage:
def main():
    linked_list = LinkedList()

    # Adding elements at the beginning of the linked list
    linked_list.add_first(30)
    linked_list.add_first(20)
    linked_list.add_first(10)

    print("Size of the linked list:", linked_list.get_size())
```

```
item_to_search = 20
if linked_list.search(item_to_search):
    print(f'{item_to_search} found in the linked list.')
else:
    print(f'{item_to_search} not found in the linked list.')

if __name__ == "__main__":
    main()
```

Put Your Solution Here:

14. In a circular linked list, how do you know that you have reached the end of the list during traversal?
- a) The next pointer of the last node points to None.
 - b) The next pointer of the last node points to the first node.
 - c) The previous pointer of the last node points to None.
 - d) The last node has a special "end" flag.

Put Your Solution Here:

-
15. Which of the following is NOT a valid way to reverse a singly linked list? Justify your answer.
- a) Iterative approach
 - b) Recursive approach
 - c) Using a stack
 - d) Using a queue

Put Your Solution Here:

Justification: